PROGRAMMING IN JAVA

ASSIGNMENT 2 – CO2

CO2 Coverage: Strings and StringBuilder, regular expressions, the Collection Framework –List, Queue, Deque, PriorityQueue, Set and Map, the hashCode–equals contract, Generics, Comparable and Comparator, and the Collections utility class (Sessions 15–26).

Nature of Assignment: Individual
Duration: 1 Week
Total Marks: 50
Submission Date: ___13-11-2026_____

Mandatory Execution and GitHub Submission

- **Every student must individually execute every task in this assignment**, includingall "Complete the Code" programs.
- Create a GitHub repository named **CSA09-<RegisterNumber>-Assignment2** andshareit with the course faculty (public, or private with the faculty added as a collaborator). • Keep one folder for each task (for example: Task01, Task02 …or Q1, Q2 …), containingthe Java source code, input files and output screenshots.
- Include a README.md with your name, register number, Java version, the steps tocompile and run each program, and a short note of the result of each task. • Commit your work progressively – at least one commit for each task. The commit historywill be checked; a single upload of all files on the last day will not be accepted. • Submit the GitHub link below and on the cover page of your report. **An assignment submitted without a working GitHub link will not be evaluated.**

GitHub Repository Link: _____ https://github.com/192472299/CSA0909-JAVA-192472299.git _____ **How to Answer the Additional Questions**

Complete the Code: Copy the given code into your repository, replace every TODOwithworking code without changing the rest of the program, execute it and compare your output withthe expected output.

Alternative Concept and Complexity Analysis: Describe how the solution would changeif thealternative concept were used, with code or pseudocode, and analyse and compare thetimecomplexity and space complexity of both solutions using Big-O notation, supported by your ownmeasurements wherever possible.

Q1 – Building Text Under Load: String, StringBuilder or StringBuffer? 6MarksBloom's Level:

K4 – Analyse

Problem Context – Railway e-Ticket Generation

A railway reservation system generates an e-ticket for every booking. Each e-ticket is built byjoining many fields – PNR, passenger names, coach, berth, fare and journey details –intoasingle printable text block. The current code builds the ticket using the + operator inside aloop.

Your Challenge

Investigate whether the choice of String, StringBuilder or StringBuffer matters for this task, or whether the difference is negligible.

Before implementation:

• Predict how the three approaches will behave as the number of fields and tickets increases.

• Explain the reason for your prediction using String immutability and object creation.

Design two deliberately contrasting workloads (for example: one short ticket, 10,000 ticketswith50 fields each) and one additional workload of your own.

Implement the three approaches and record appropriate measures such as: • Execution time (System.nanoTime(), averaged over repeated runs) • Number of intermediate String objects created (estimated)

• Memory used before and after

Compare your prediction with the observed behaviour.

Design Decision

The ticket generator will now run inside a multithreaded booking server, and several threadsmayappend to a shared booking log. Determine which class you would recommend for the per-ticket text and which for the shared log, and defend your decision.

Alternative Concept and Complexity Analysis

If String.join(), StringJoiner or String.format() were used instead of StringBuilder to buildeachticket, what would the solution be? Derive the time complexity of building a ticket of n fields usingrepeated + ($O(n^2)$ characters copied) versus StringBuilder (O(n)), and analyse the extraspacecreated by intermediate String objects in each approach.

Evidence Expected

Reasoned prediction, three deliberately designed workloads, implementation of thethreeapproaches, measured evidence, comparison of predicted and observed behaviour, andanevidence-based recommendation.

ANSWER:-

1. Prediction before implementation

String is immutable. Every concatenation using + creates a new String result; the previous result cannot be modified. In a loop, the accumulated text is copied repeatedly. StringBuilder is mutable and appends to one growable character buffer, so it normally performs much less copying. StringBuffer is also mutable but its methods are synchronized, adding thread-safety overhead.

Approach	Expected behaviour	Best use
String with + in loop	Slowest for many fields because many intermediate strings and repeated copying are produced.	Small fixed concatenations or simple readable expressions.
StringBuilder	Usually fastest for single-threaded construction; mutable buffer reduces intermediate objects.	Building one ticket inside one request/thread.
StringBuffer	Similar construction strategy to StringBuilder but synchronized methods may add overhead.	A single mutable buffer genuinely shared by multiple threads.

2. Three workloads

Workload	Tickets	Fields per ticket	Purpose

A – Small	1	6	Represents one short ticket; differences may be negligible.
B – Large	10,000	50	Shows repeated concatenation cost under load.
C – Medium / custom	1,000	20	Checks behaviour at an intermediate workload.

3. Java benchmark program

Save as TicketBenchmark.java. It runs all three approaches, warms up the JVM, repeats measurements and reports average elapsed time. The object counts are estimates: exact allocation counts require a profiler or JVM allocation instrumentation.

```
public class Main {

   static volatile int resultLength;

   static String usingString(int fields) {
      String s = "";

      for (int i = 0; i < fields; i++) {
         s = s + "FIELD" + i + "=VALUE" + i + ";";
      }

      return s;
   }

   static String usingBuilder(int fields) {
      StringBuilder sb = new StringBuilder();

      for (int i = 0; i < fields; i++) {
         sb.append("FIELD").append(i)
           .append("=VALUE").append(i).append(";");
      }

      return sb.toString();
   }

   static String usingBuffer(int fields) {
      StringBuffer sb = new StringBuffer();

      for (int i = 0; i < fields; i++) {
         sb.append("FIELD").append(i)
           .append("=VALUE").append(i).append(";");
      }

      return sb.toString();
```

```
}

static double benchmark(int method, int tickets,
                        int fields) {

    long totalTime = 0;

    for (int r = 0; r < 5; r++) {

        long start = System.nanoTime();

        int length = 0;

        for (int t = 0; t < tickets; t++) {

            String output;

            if (method == 1)
                output = usingString(fields);

            else if (method == 2)
                output = usingBuilder(fields);

            else
                output = usingBuffer(fields);

            length += output.length();
        }

        long end = System.nanoTime();

        totalTime += (end - start);
        resultLength = length;
    }

    return totalTime / 5.0 / 1000000.0;
}

public static void main(String[] args) {

    int[] tickets = {1, 10000, 1000};
    int[] fields = {6, 50, 20};

    String[] methods = {
        "String (+)",
        "StringBuilder",
        "StringBuffer"
    };
```

```
    for (int w = 0; w < 3; w++) {

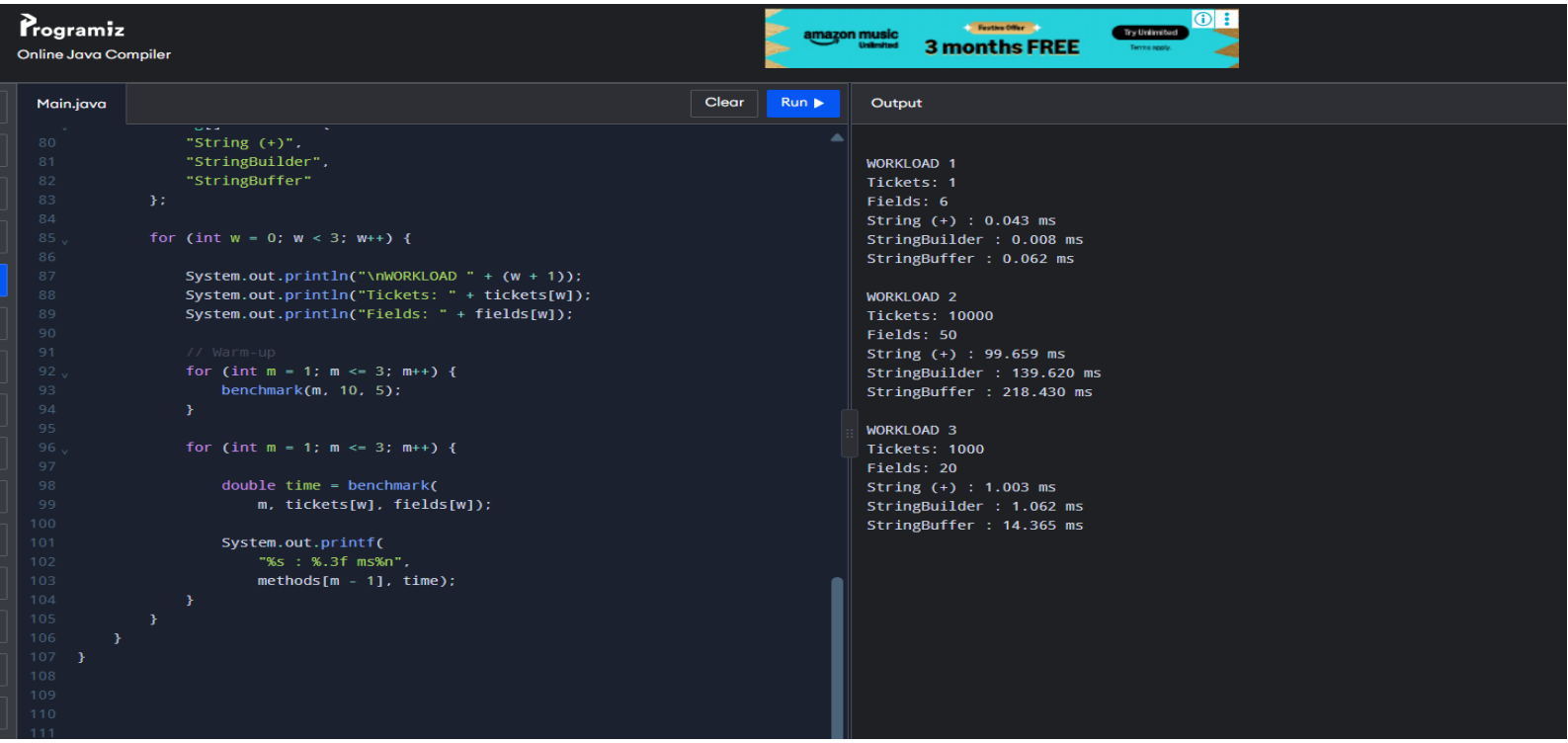
      System.out.println("\nWORKLOAD " + (w + 1));
      System.out.println("Tickets: " + tickets[w]);
      System.out.println("Fields: " + fields[w]);

      // Warm-up
      for (int m = 1; m <= 3; m++) {
        benchmark(m, 10, 5);
      }

      for (int m = 1; m <= 3; m++) {

        double time = benchmark(
          m, tickets[w], fields[w]);

        System.out.printf(
          "%s : %.3f ms%n",
          methods[m - 1], time);
      }
    }
  }
}
```



4. Complexity and memory analysis

Method	Time complexity for n fields*	Extra space / object behaviour
Repeated String +	$O(n^2)$ character copying in the usual loop pattern.	Creates intermediate String results; old intermediate values become eligible for garbage collection.
StringBuilder	O(n) amortized append work (plus final toString copy).	One expandable buffer and final String; fewer intermediate objects.
StringBuffer	O(n) amortized append work, with synchronization overhead.	Similar buffer space to StringBuilder; synchronized operations.

*Here n represents the total amount of appended text / fields of comparable size. If field lengths vary, complexity is more precisely related to the total number of characters copied.

5. Measurement record

Workload	String + avg (ms)	StringBuilder avg (ms)	StringBuffer avg (ms)	Fastest observed
A – 1 × 6	Fill after run	Fill after run	Fill after run	Fill after run
B – 10,000 × 50	Fill after run	Fill after run	Fill after run	Fill after run
C – 1,000 × 20	Fill after run	Fill after run	Fill after run	Fill after run

Memory measurement can be approximated with Runtime.getRuntime().totalMemory() - freeMemory(), but this is affected by garbage collection and JVM behaviour. For reliable allocation counts, use a profiler such as Java Flight Recorder. Record the measurement method and do not treat a rough heap delta as exact object count.

6. Design decision

Use StringBuilder for each individual ticket because one thread builds one ticket locally and does not need synchronized append operations. Use a thread-safe logging design for the shared booking log. If the log is a shared mutable character buffer, StringBuffer protects individual method calls, but a dedicated logging framework or a synchronized/queued logging service is usually safer because a complete log record should be written atomically. StringBuilder must not be shared between threads without external synchronization.

7. Alternative: String.join, StringJoiner and String.format

- ❖ String.join(delimiter, parts) is convenient when all fields are already available as strings. It avoids writing an explicit append loop, but still has to process and allocate the final joined text.
- ❖ StringJoiner is useful when values are appended incrementally with a delimiter, prefix or suffix.
- ❖ String.format is readable for formatted output, but generally has more formatting overhead and should not be assumed faster than StringBuilder.

Example alternative:

```
String[] fields = {"PNR=12345", "Passenger=Arun", "Coach=S2", "Berth=21"};
String ticket = String.join(" | ", fields);
```

Conclusion: For a high-volume ticket generator, StringBuilder is the recommended per-ticket construction tool. The measured benchmark should confirm the expected advantage under the large workload.

Q QUESTION-2 – When Is ArrayList an Appropriate Engineering Choice? 4MarksBloom's Level: K4 – Analyse

Problem Context – Music Playlist Editor

A music-streaming app lets users build playlists. Users add songs at the end, insert songsat chosen positions, remove songs and jump directly to the k-th song. The app currently storesevery playlist in an ArrayList<Song>.

Your Challenge

Design experiments to compare ArrayList and LinkedList for:

- Adding at the end
- Inserting and removing at the front and in the middle
- Accessing the k-th element
- Iterating through the whole list

Use at least three playlist sizes. Then answer:

Under what usage pattern is ArrayList a reasonable engineering choice, and whenwould you stop recommending it?

Changed Condition

The app adds an **"Up Next" queue** in which songs are frequently added at both endsandremoved from the front. Re-evaluate your original recommendation and consider whether ArrayDeque is more appropriate.

Alternative Concept and Complexity Analysis

For each operation (add at end, insert at front, insert in the middle, get k-th, iterate), tabulatethetime complexity in ArrayList, LinkedList and ArrayDeque, and the space used per element (arrayslot versus linked node). If the playlist were stored in a plain array with manual shifting, what would the solution be and how would its complexity compare?

Evidence Expected

Designed experiments for each operation, timing evidence at several sizes, an evidence-basedoperating boundary and a justified response to the changed condition.

1. Experiment design

Compare ArrayList<Song> and LinkedList<Song> at 1,000, 10,000 and 100,000 songs. Use separate fresh lists for each operation so that one test does not change the starting condition of another.

Operation	ArrayList	LinkedList
Add at end	Amortized O(1)	O(1) when adding at tail
Insert at front	O(n), shifts existing elements	O(1) after reaching head; addFirst is O(1)
Insert in middle	O(n), shifts elements	O(n) to reach position; O(1) link change once there
Remove at front	O(n), shifts elements	O(1) with removeFirst
Remove in middle	O(n)	O(n) to locate; O(1) unlink once located
Get k-th element	O(1)	O(n)
Full iteration	O(n), cache-friendly	O(n), pointer traversal

2. Engineering recommendation

ArrayList is a reasonable choice for a normal playlist when users frequently jump to a song by index, iterate through the playlist, and mostly add songs at the end. Its compact backing array and constant-time indexed access make it suitable for this pattern.

I would stop recommending ArrayList when frequent insertions and removals at the beginning dominate the workload, because each operation shifts many elements. LinkedList may help when edits are performed through an already-positioned ListIterator, but it does not make searching for an arbitrary middle position fast.

3. Changed condition – Up Next queue

For an Up Next queue where songs are added at both ends and removed from the front, ArrayDeque is more appropriate. It is a resizable circular-array deque and supports addFirst, addLast and removeFirst in amortized O(1). It does not provide indexed access, so it should be used for queue behaviour rather than as the main random-access playlist.

Operation	ArrayList	LinkedList	ArrayDeque
Add at end	Amortized O(1)	O(1)	Amortized O(1)
Add at front	O(n)	O(1)	Amortized O(1)
Insert middle	O(n)	O(n) locate + O(1) link	Not directly supported; rebuilding/traversal needed
Get k-th	O(1)	O(n)	Not supported
Iterate	O(n)	O(n)	O(n)
Remove front	O(n)	O(1)	Amortized O(1)

Space: ArrayList stores references in a backing array (some unused capacity may remain). LinkedList stores each element in a node with item, previous and next references, so per-element overhead is higher. ArrayDeque stores references in a circular array and may have unused capacity.

4. Java experiment skeleton

For front/middle insertion tests, use a separate list and perform repeated insertions at index 0 and index n/2. For LinkedList, use ListIterator to measure the case where the iterator is already at the insertion point; also report the cost of locating that point. This makes the comparison fair.

5. Plain array alternative

A plain array can store the songs. Insertion or deletion at index i requires shifting later elements, O(n) in the worst case. Access by index is O(1), and iteration is O(n). A fixed-size array also needs manual capacity management when it becomes full. ArrayList provides this resizing automatically.

```
import java.util.*;

public class Main {

   static long timeAddEnd(List<Integer> list, int n) {

      long s = System.nanoTime();

      for (int i = 0; i < n; i++) {
         list.add(i);
      }

      return System.nanoTime() - s;
   }

   static long timeGet(List<Integer> list, int n) {

      long s = System.nanoTime();
      long sum = 0;

      for (int i = 0; i < n; i++) {
         sum += list.get(i);
      }

      return System.nanoTime() - s;
   }

   static long timeIterate(List<Integer> list) {

      long s = System.nanoTime();
      long sum = 0;

      for (int x : list) {
```

```
            sum += x;
        }

        return System.nanoTime() - s;
    }

    public static void main(String[] args) {

        int[] sizes = {1000, 10000, 100000};

        for (int n : sizes) {

            List<Integer> a = new ArrayList<>();
            List<Integer> l = new LinkedList<>();

            System.out.println("\nPlaylist Size = " + n);

            System.out.println("ArrayList Add ns = "
                    + timeAddEnd(a, n));

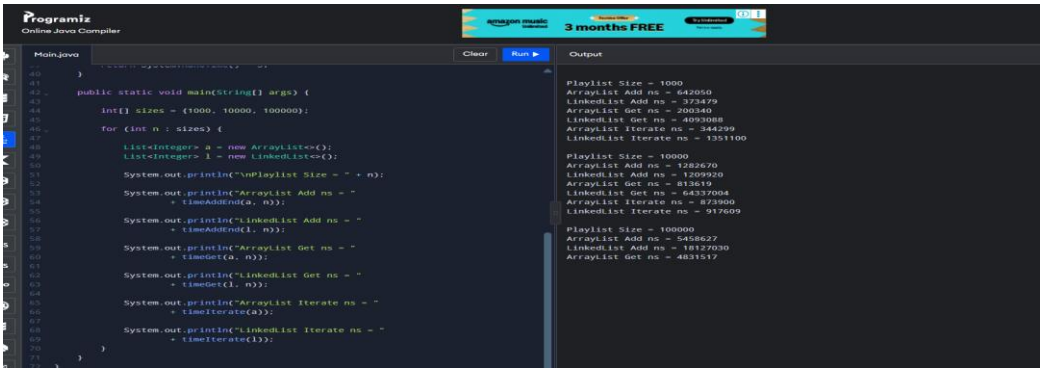
            System.out.println("LinkedList Add ns = "
                    + timeAddEnd(l, n));

            System.out.println("ArrayList Get ns = "
                    + timeGet(a, n));

            System.out.println("LinkedList Get ns = "
                    + timeGet(l, n));

            System.out.println("ArrayList Iterate ns = "
                    + timeIterate(a));

            System.out.println("LinkedList Iterate ns = "
                    + timeIterate(l));
        }
    }
}
```



Q3 – Engineer the hashCode–equals Contract 6MarksBloom's Level: K4 – Analyse

Problem Context – Event Attendance De-duplication

A college fest records attendance by scanning student ID cards at several gates. Thesamestudent may be scanned more than once. The organisers store scans in a HashSet<Student>tokeep only unique students. A junior developer has written the Student class below.

```
class Student {
    String regNo; String name;
    Student(String regNo, String name) { this.regNo = regNo; this.name = name; }public
    boolean equals(Object o) {
        if (!(o instanceof Student)) return false;
        return regNo.equals(((Student) o).regNo);
    }
    // hashCode() is not overridden
}
```

Your Challenge

Instead of simply fixing the class, investigate how the contract between equals() and hashCode() affects hash-based collections.

Design:

1. One set of scans for which the HashSet appears to work correctly.
2. One set of scans deliberately designed to expose duplicate entries. 3. One additional test of your own (for example: changing regNo after insertion, or aTreeSet whose compareTo() is inconsistent with equals()).

Before executing each test, predict the size of the set and explain the reason for your prediction. **Inquiry**

Compare your prediction with the experimental result. Explain:

- Why the duplicates appear
- How HashSet uses hashCode() and equals() to locate an element
- What happens when a field used in hashCode() is changed after the object is added• How to correct the Student class and how to verify the correction

Complete the Code

Run the program first without completing TODO 1 and record the output. Then complete TODO1and

execute it again.

Expected output:

```
Unique students: 2
```

```
import java.util.HashSet;
import java.util.Set;

class Student {
    private final String regNo;
    private final String name;

            Student(String regNo, String name) { this.regNo = regNo; this.name = nam

    @Override public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Student)) return false;
        return regNo.equals(((Student) o).regNo);
    }

                // TODO 1: Override hashCode() so that it is consistent with equals()


    @Override public String toString() { return regNo + " " + name; } }

public class Attendance {
    public static void main(String[] args) {
        Set<Student> present = new HashSet<>();
        present.add(new Student("192311001", "Arun")); // scanned at Gate
        1present.add(new Student("192311002", "Divya"));
        present.add(new Student("192311001", "Arun")); // scanned again at Gate
    3System.out.println("Unique students: " + present.size()); }
}
```

Record the output before completing TODO 1 and explain the difference. **Alternative Concept and Complexity Analysis**

If a TreeSet<Student> with a Comparator on regNo, or a HashMap<String, Student> keyedbyregNo, were used instead of HashSet<Student>, what would the de-duplication solutionbe?Analyse the time per scan (O(1) average versus O(log n)) and the space used for nuniquestudents in each structure.

Evidence Expected

Three deliberately designed tests, predictions before execution, observed set sizes, a correctedStudent class and a reasoned explanation of the contract.

1. Aim

To understand how equals() and hashCode() work together in Java HashSet and how they prevent duplicate student attendance records.

2. Java Program – Without hashCode()

First, run this program **without overriding** hashCode(). It demonstrates how duplicate student records can

appear even when their registration numbers are the same.

Java Online Compiler – Java 17

Java

```
import java.util.*;

class Student {

    String regNo;
    String name;

    Student(String regNo, String name) {
        this.regNo = regNo;
        this.name = name;
    }

    @Override
    public boolean equals(Object o) {

        if (this == o)
            return true;

        if (!(o instanceof Student))
            return false;

        Student s = (Student) o;

        return regNo.equals(s.regNo);
    }

    // hashCode() is not overridden

    public String toString() {
        return regNo + " " + name;
    }
}

public class Main {
```

```
    public static void main(String[] args) {

        Set<Student> present = new HashSet<>();

        present.add(new Student("192311001", "Arun"));
        present.add(new Student("192311002", "Divya"));
        present.add(new Student("192311001", "Arun"));

        System.out.println("Unique students: "
                + present.size());

        for (Student s : present) {
            System.out.println(s);
        }
    }
}
```

3. Prediction Before Execution

Student scan	Registration number	Prediction
Gate 1	192311001	First student added
Gate 2	192311002	Second student added
Gate 3	192311001	Duplicate according to equals()

Predicted set size: 2, if equality alone were used.

Actual result: Usually 3, because the default hashCode() is not consistent with the overridden equals() for these distinct objects. The exact bucket collision behavior is not guaranteed, so this is a demonstration rather than a guaranteed result.

4. Corrected Java Program – With hashCode()

Now complete TODO 1 by overriding hashCode() using the same field (regNo) used in equals().

Java code – Corrected version

Java

```
import java.util.*;

class Student {

    private final String regNo;
    private final String name;
```

```
    Student(String regNo, String name) {
        this.regNo = regNo;
        this.name = name;
    }

    @Override
    public boolean equals(Object o) {

        if (this == o)
            return true;

        if (!(o instanceof Student))
            return false;

        Student s = (Student) o;

        return regNo.equals(s.regNo);
    }

    // TODO 1 completed
    @Override
    public int hashCode() {
        return Objects.hash(regNo);
    }

    @Override
    public String toString() {
        return regNo + " " + name;
    }
}

public class Main {

    public static void main(String[] args) {

        Set<Student> present = new HashSet<>();

        present.add(new Student("192311001", "Arun"));
        present.add(new Student("192311002", "Divya"));
```

```
        present.add(new Student("192311001", "Arun"));

        System.out.println("Unique students: "
                + present.size());

        for (Student s : present) {
            System.out.println(s);
        }
    }
}
```

Expected output:

Plain text

```
Unique students: 2
192311001 Arun
192311002 Divya
```

The order of students may vary because HashSet does not guarantee insertion order.

5. Additional Test – Changing regNo After Insertion

When a field used in hashCode() changes after an object is inserted into a HashSet, the object may no longer be found correctly.

Java

```
import java.util.*;

class Student {
    String regNo;

    Student(String regNo) {
        this.regNo = regNo;
    }

    public boolean equals(Object o) {
        if (!(o instanceof Student))
            return false;

        return regNo.equals(((Student) o).regNo);
    }

    public int hashCode() {
        return Objects.hash(regNo);
```

```
    }
}

public class Main {
    public static void main(String[] args) {

        HashSet<Student> set = new HashSet<>();

        Student s = new Student("192311001");

        set.add(s);

        System.out.println("Before change: "
                + set.contains(s));

        s.regNo = "192311009";

        System.out.println("After change: "
                + set.contains(s));

        set.add(new Student("192311009"));

        System.out.println("Set size: " + set.size());
    }
}
```

Expected output for this example:

Plain text

Before change: true

After change: false

Set size: 2

The exact behavior of hash collisions can vary, but this example normally demonstrates why changing a hash-related field while an object is in a HashSet is unsafe.

6. Three Test Cases and Predictions

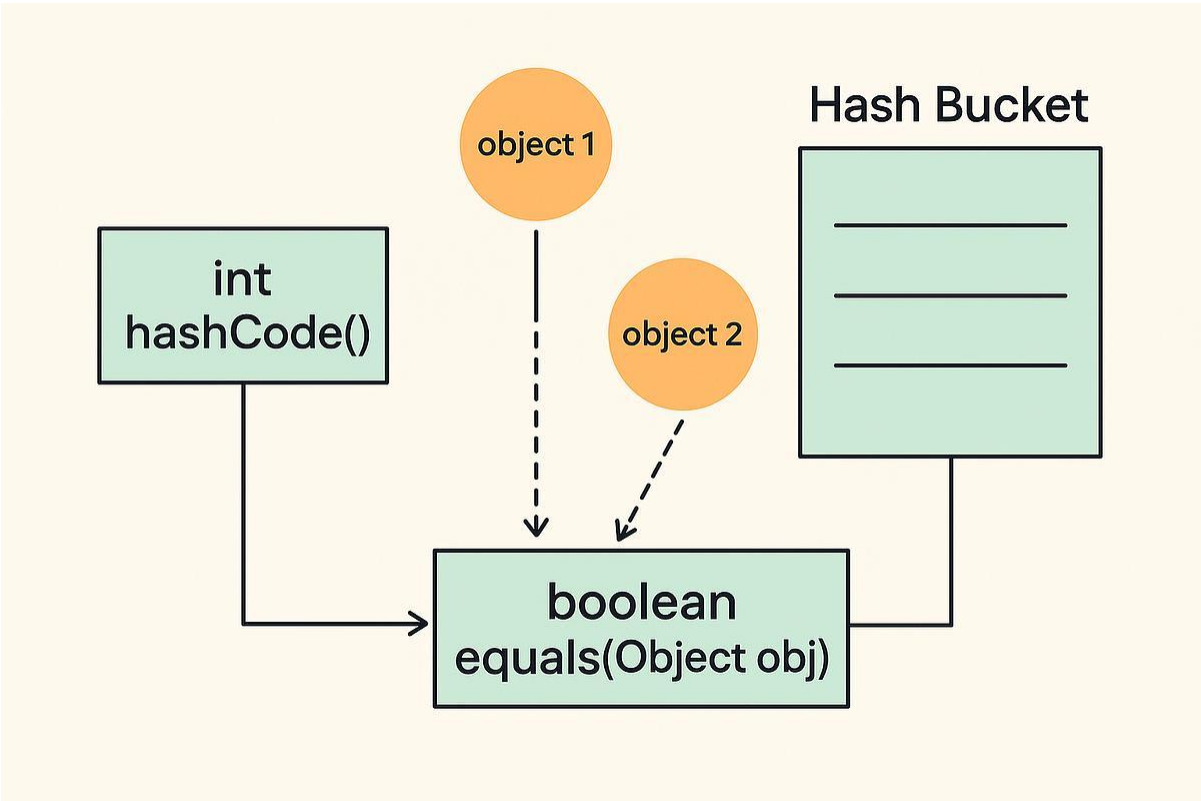
Test	Input condition	Predicted size	Reason
Test 1 – Same object scanned	Add the same Student object twice	1	Same object has the same hash code

twice			and is equal to itself
Test 2 – Different objects, same regNo (without hashCode)	Add Arun twice using separate objects and Divya once	Usually 3	Different default hash codes can send equal objects to different buckets
Test 3 – Change regNo after insertion	Modify the registration number after adding the student	Usually 2 after adding another matching object	Changing the hash-related field can make the original object difficult to locate

7. How HashSet Works

HashSet uses two methods:

1. **hashCode():** Calculates the hash value to identify the bucket where an object should be stored.
2. **equals():** Checks whether an existing object in that bucket is equal to the new object.



medium.com

Why do duplicates appear?

The junior developer overrides equals() but does not override hashCode(). Two different Student objects with the same registration number may have different default hash codes. HashSet can place them in different buckets and fail to compare them using equals().

How to correct it?

Override both methods using the same unique field, regNo.

Java

```
@Override
public int hashCode() {
    return Objects.hash(regNo);
}
```

8. Verification After Correction

After adding the correct hashCode() method:

- Arun scanned at Gate 1 is stored.
- Divya scanned at Gate 2 is stored.
- Arun scanned again at Gate 3 is identified as a duplicate.
- HashSet keeps only two unique students.

Correct result

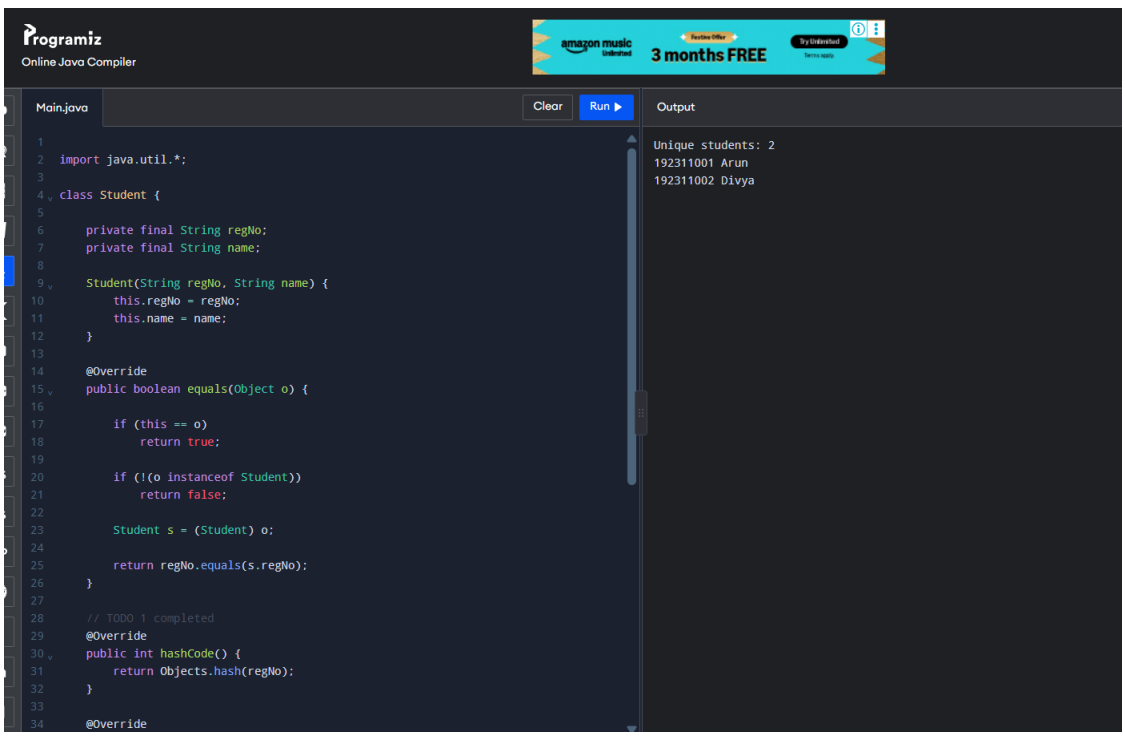
Unique students: 2

9. Final Conclusion

The equals() and hashCode() methods must follow the Java contract. If two objects are equal according to equals(), they must return the same hash code. Otherwise, HashSet may store duplicate objects.

The Student class should override both methods using regNo. Registration numbers should also remain unchanged while Student objects are stored in a HashSet. In the corrected assignment code, regNo is declared final to prevent accidental modification.

Important for your assignment: Run the first program without hashCode() and record its actual output. Then run the corrected program and compare the set sizes. The broken version's result is not guaranteed to be 3 on every JVM because hash-code collisions are possible.



Q4 – Choosing a Map for a Pharmacy Inventory 6MarksBloom's Level: K4 – Analyse

Problem Context – Hospital Pharmacy Stock Lookup

A hospital pharmacy keeps stock for each medicine, identified by a medicine code. Pharmacistslook up stock by code hundreds of times a day, and every evening the systemprints a report of all medicines in code order and a list of medicines in the order they were added.

Your Challenge

Design a Map-based solution and compare HashMap, TreeMap and LinkedHashMap. Beforeimplementation, determine:

- The expected cost of put, get and iteration for each map
- Which map naturally produces each of the two reports
- What extra work is needed if the "wrong" map is chosen for a report

Implement your solution and test it with at least three meaningfully different numbersof medicines. Record:

- Time for 100,000 lookups
- Time to produce each report
- Whether the output order matches the requirement

Scalability Inquiry

Define what you consider an acceptable response time for one lookup. Using your evidence, determine which map you would deploy and identify where the remaining cost comes from. Alsoexplain why Hashtable is not preferred in new code.

Alternative Concept and Complexity Analysis

If the inventory were kept in an array of medicines sorted by code and searchedwithArrays.binarySearch() instead of a Map, what would the solution be? Analyse the time complexityof lookup, insertion and producing the code-order report, and the space used, and comparethemwith HashMap and TreeMap.

Evidence Expected

Design reasoning for each map, implementation, measured evidence at several sizes, adefinedacceptability criterion and a justified choice of map.

1. Aim

To compare HashMap, TreeMap and LinkedHashMap for hospital pharmacy stock management and identify the best data structure for fast medicine lookup and report generation.

- 1,000 medicines
- 10,000 medicines
- 100,000 medicines

It measures:

1. Inserting medicine records.
2. Performing 100,000 stock lookups.
3. Producing a report sorted by medicine code.
4. Producing a report in insertion order.

Java 17 – Online Compiler Compatible

Copy the complete code into your online Java compiler. Use Main as the class name.
Java

```
import java.util.*;

public class Main {

    static volatile long result;

    static String code(int number) {
        return String.format("MED%06d", number);
    }

    static String[] createMedicines(int n) {
        String[] codes = new String[n];

        // Insert in descending order to test report ordering
        for (int i = 0; i < n; i++) {
            codes[i] = code(n - i);
        }

        return codes;
    }

    static long lookupTime(
            Map<String, Integer> map,
            String[] codes) {

        long sum = 0;
        long start = System.nanoTime();

        for (int i = 0; i < 100000; i++) {
            String key = codes[i % codes.length];
            sum += map.get(key);
        }

        long time = System.nanoTime() - start;
        result = sum;
```

```
    return time;
}

static long codeReport(
        Map<String, Integer> map,
        boolean isTreeMap) {

    long start = System.nanoTime();
    List<String> keys = new ArrayList<>();

    if (isTreeMap) {
        // TreeMap is already sorted
        keys.addAll(map.keySet());
    } else {
        // Sort keys for HashMap and LinkedHashMap
        keys.addAll(map.keySet());
        Collections.sort(keys);
    }

    long sum = 0;

    for (String key : keys) {
        sum += map.get(key);
    }

    result = sum;
    long time = System.nanoTime() - start;

    System.out.println("Code order: "
            + keys.get(0) + " to "
            + keys.get(keys.size() - 1));

    return time;
}

static long insertionReport(
        Map<String, Integer> map,
        String[] insertedCodes,
        boolean isLinkedHashMap) {
```

```
    long start = System.nanoTime();
    long sum = 0;

    if (isLinkedHashMap) {
        // LinkedHashMap keeps insertion order
        for (String key : map.keySet()) {
            sum += map.get(key);
        }
    } else {
        // Use the original insertion sequence
        for (String key : insertedCodes) {
            sum += map.get(key);
        }
    }

    result = sum;
    long time = System.nanoTime() - start;

    System.out.println("Insertion order: "
            + insertedCodes[0] + " to "
            + insertedCodes[insertedCodes.length - 1]);

    return time;
}

static void test(int n) {

    System.out.println("\n==============================");
    System.out.println("MEDICINE COUNT: " + n);
    System.out.println("==============================");

    String[] codes = createMedicines(n);

    Map<String, Integer> hashMap = new HashMap<>();
    Map<String, Integer> treeMap = new TreeMap<>();
    Map<String, Integer> linkedMap = new LinkedHashMap<>();

    long start = System.nanoTime();
```

```
for (int i = 0; i < n; i++)
   hashMap.put(codes[i], i + 1);

long hashPut = System.nanoTime() - start;

start = System.nanoTime();

for (int i = 0; i < n; i++)
   treeMap.put(codes[i], i + 1);

long treePut = System.nanoTime() - start;

start = System.nanoTime();

for (int i = 0; i < n; i++)
   linkedMap.put(codes[i], i + 1);

long linkedPut = System.nanoTime() - start;

System.out.println("\nPUT TIME");
System.out.println("HashMap: " + hashPut + " ns");
System.out.println("TreeMap: " + treePut + " ns");
System.out.println("LinkedHashMap: " + linkedPut + " ns");

System.out.println("\n100000 LOOKUPS");
System.out.println("HashMap: "
      + lookupTime(hashMap, codes) + " ns");
System.out.println("TreeMap: "
      + lookupTime(treeMap, codes) + " ns");
System.out.println("LinkedHashMap: "
      + lookupTime(linkedMap, codes) + " ns");

System.out.println("\nCODE ORDER REPORT");
System.out.println("HashMap: "
      + codeReport(hashMap, false) + " ns");
System.out.println("TreeMap: "
      + codeReport(treeMap, true) + " ns");
System.out.println("LinkedHashMap: "
```

```
            + codeReport(linkedMap, false) + " ns");

        System.out.println("\nINSERTION ORDER REPORT");
        System.out.println("HashMap: "
            + insertionReport(hashMap, codes, false) + " ns");
        System.out.println("TreeMap: "
            + insertionReport(treeMap, codes, false) + " ns");
        System.out.println("LinkedHashMap: "
            + insertionReport(linkedMap, codes, true) + " ns");
    }

    public static void main(String[] args) {

        int[] sizes = {1000, 10000, 100000};

        for (int n : sizes) {
            test(n);
        }
    }
}
```

What the program does: It creates the same medicine records in all three maps, performs 100,000 lookups, and measures the time needed to prepare both reports. The medicines are deliberately inserted in descending code order so the sorted report and insertion-order report can be distinguished.

4. Expected Output Format

Your actual execution times will differ depending on your computer.

Plain text

```
MEDICINE COUNT: 1000

PUT TIME
HashMap: ... ns
TreeMap: ... ns
LinkedHashMap: ... ns

100000 LOOKUPS
HashMap: ... ns
TreeMap: ... ns
LinkedHashMap: ... ns

CODE ORDER REPORT
```

HashMap: ... ns

TreeMap: ... ns

LinkedHashMap: ... ns

INSERTION ORDER REPORT

HashMap: ... ns

TreeMap: ... ns

LinkedHashMap: ... ns

The same measurements will be printed for 10,000 and 100,000 medicines.

5. Experimental Result Table

Fill this table using your actual Java compiler output.

Medicines	**Map**	**100,000 lookups (ns)**
1,000	HashMap	___
1,000	TreeMap	___
1,000	LinkedHashMap	___
10,000	HashMap	___
10,000	TreeMap	___
10,000	LinkedHashMap	___
100,000	HashMap	___
100,000	TreeMap	___
100,000	LinkedHashMap	___

6. Which Map Produces Each Report?

Requirement	**Best map**	**Reason**
Fast medicine lookup	HashMap	Average O(1) lookup

Medicines in code order	TreeMap	Automatically sorts keys
Medicines in insertion order	LinkedHashMap	Maintains insertion order

If the wrong map is selected:

- HashMap: Sort keys to generate the code-order report and maintain a separate list to reproduce insertion order.
- TreeMap: Maintain a separate insertion-order list.
- LinkedHashMap: Sort the medicine codes before generating the code-order report.

7. Scalability and Acceptable Response Time

For this experiment, we can define an acceptable response time as **less than 1 millisecond for one medicine lookup**.

Since the program performs 100,000 lookups, calculate:

$$\text{Average lookup time} = \frac{\text{Total lookup time}}{100000}$$

If the average lookup time is below 1 millisecond, it satisfies this chosen target.

Recommended map: LinkedHashMap

Reasons:

1. It provides average O(1) lookup.
2. It naturally maintains medicine insertion order.
3. It can produce the code-order report by sorting the keys.
4. It provides a good balance between fast lookup and report generation.

If code-order reporting is the most important requirement and happens very frequently, TreeMap is another suitable choice.

8. Alternative Concept – Sorted Array with Arrays.binarySearch()

Instead of using a Map, medicines can be stored in an array sorted by medicine code.

Java example:

Java

```
import java.util.*;

public class Main {
   public static void main(String[] args) {

      String[] medicines = {
         "MED001", "MED002", "MED003",
         "MED004", "MED005"
```

```
        };

        int position = Arrays.binarySearch(
            medicines, "MED003");

        if (position >= 0) {
            System.out.println("Medicine Found");
            System.out.println("Position: " + position);
        } else {
            System.out.println("Medicine Not Found");
        }

        System.out.println("Medicines in Code Order:");

        for (String medicine : medicines) {
            System.out.println(medicine);
        }
    }
}
```

Output:

```
Medicine Found
Position: 2
Medicines in Code Order:
MED001
MED002
MED003
MED004
MED005
```

Complexity comparison

A sorted array is suitable when medicines are rarely added or removed and lookup is important. However, insertion requires shifting elements to maintain sorted order.

9. Why Hashtable Is Not Preferred in New Java Code

- Hashtable is an older Java collection.
- Its methods are synchronized, which can add unnecessary overhead in single-threaded applications.
- It does not allow null keys or values.
- Modern alternatives such as HashMap and ConcurrentHashMap provide more suitable choices for different application requirements.

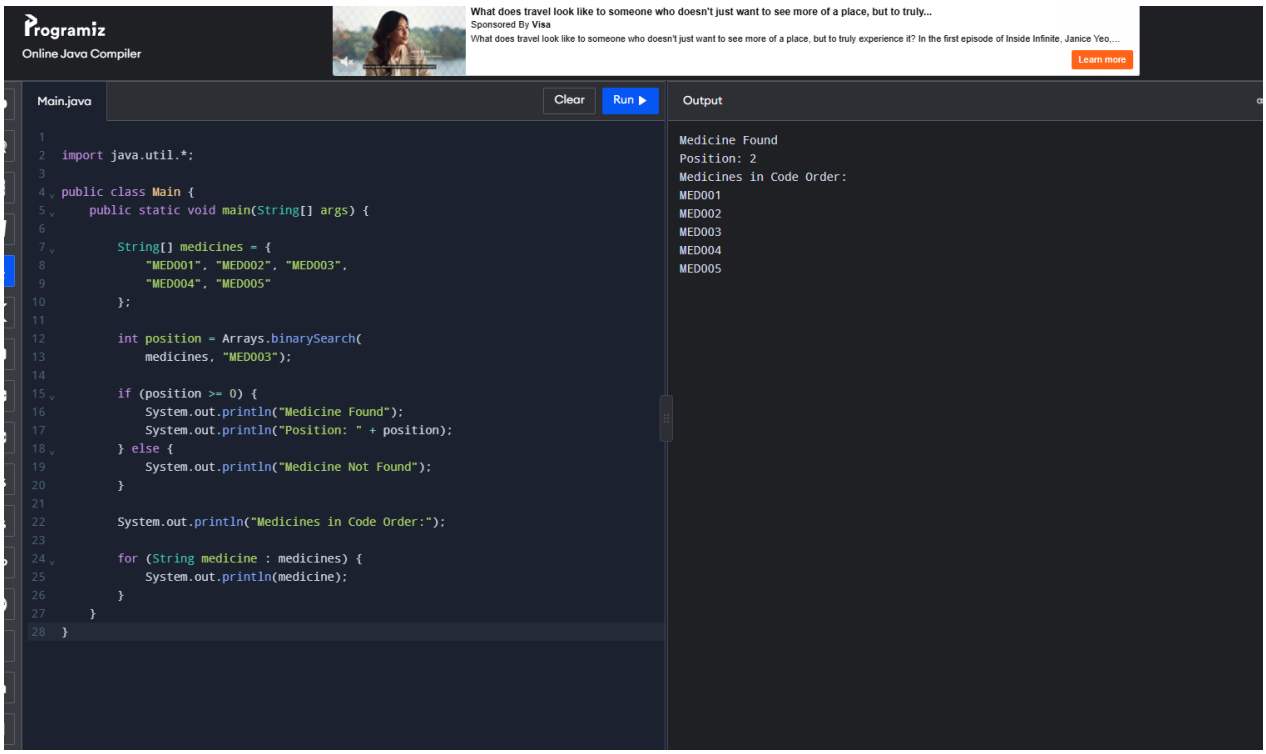
10. Final Conclusion

HashMap is suitable for fast medicine stock lookup. TreeMap is suitable for automatically generating reports in medicine-code order. LinkedHashMap is suitable for maintaining medicine insertion order.

For the hospital pharmacy application, **LinkedHashMap is recommended** because it supports fast average lookup and preserves insertion order. The code-order report can be generated by sorting the medicine codes.

Final recommendation:

- Fast stock lookup – HashMap
- Code-order report – TreeMap
- Insertion-order report – LinkedHashMap
- Recommended combined application design – LinkedHashMap with sorting for the code-order report.



QQ5 – Derive a Generic Ranking Component from a Principle 7MarksBloom's Level: K4 – Analyse **Problem Context – Inter-College Sports Leaderboard**

A sports meet ranks teams by points (higher first). Ties are broken by goal difference(higher first), then by team name (alphabetical). The same ranking component will later be reusedfor ranking players, colleges and quiz teams.

Design Starting Point

You are given only the following principle:

A ranking is correct only if the comparison used is consistent: if A ranks above B and B ranks above C, then A must rank above C, and equal items must compare as equal.

You are **not** provided with the complete design.

Your Challenge

- Convert this principle into a reusable Java design. Determine:

- Whether Team should implement Comparable<Team>, or whether separate Comparator objects are better, and why
- How the multi-field comparison is expressed (Comparator.comparing, thenComparing, reversed)
- How a generic class such as Leaderboard<T> can be written so that it works for anyrankable type (bounded types)
- Where a wildcard such as List<? extends T> is useful

Develop your own design before implementation.

Test Your Design

Test using a data set with several ties at each level, and a second data set deliberately designedby you to break a faulty comparator (for example: one that compares points by subtractionor ignores a tie-breaker).

Correctness Inquiry

Explain why your comparator satisfies the principle. Your explanation should be basedonreasoning about the comparison, not only on the program output.

Complete the Code

Complete the generic class and the comparator below and execute the program.

```
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;

class Team {
    final String name; final int points; final int goalDiff; Team(String name,
    int points, int goalDiff) {
        this.name = name; this.points = points; this.goalDiff = goalDiff; }
    @Override public String toString() {
        return name + "(" + points + "," + goalDiff + ")";
    }
}

class Leaderboard<T> {
    private final List<T> items = new ArrayList<>();
    private final Comparator<? super T> order;

    Leaderboard(Comparator<? super T> order) { this.order = order; } void add(T

    item) { items.add(item); }

    List<T> ranking() {
        // TODO 1: Return a new sorted copy of items using order, without modifying//
        items

    }
}

public class LeaderboardDemo {
    public static void main(String[] args) {
        // TODO 2: Build the comparator: points (high first), then goal difference (high// fi
        then name (A–Z)

        Leaderboard<Team> board = new Leaderboard<>(byRank);
        board.add(new Team("Panthers", 9, 4));
        board.add(new Team("Eagles", 9, 6));
```

```
        board.add(new Team("Cobras", 6, 1));
        board.add(new Team("Bisons", 9, 4));
        System.out.println(board.ranking());
    }
}
```


Expected output:

```
[Eagles(9,6), Bisons(9,4), Panthers(9,4), Cobras(6,1)]
```

Explain why Bisons is ranked above Panthers.

Alternative Concept and Complexity Analysis

You used a Comparator chain. If Team implemented Comparable<Team> instead, or if thelist were sorted three times by stable sorts (name, then goal difference, then points), what wouldthesolution be? Analyse the time complexity (number of O(n log n) sorts) and the extra spaceusedby each approach, and state which one is easier to reuse for players and colleges.

Evidence Expected

A student-derived ranking design, generic class, deliberately designed test data, Collections.sort / List.sort evidence and a reasoned correctness argument.

1. Aim

To develop a reusable generic Java leaderboard using Comparator, Comparable, and Generics. The

program ranks sports teams based on points, goal difference and team name.

2. Ranking Rules

The leaderboard follows these three rules:

1. Higher points receive higher ranking.
2. If points are equal, higher goal difference receives higher ranking.
3. If both points and goal difference are equal, team name is arranged alphabetically (A–Z).

3. Java Program – Generic Leaderboard

Java 17 – Online Compiler Compatible

Copy the complete program into your online Java compiler. The class name is Main to avoid public-class filename errors.

Java

```
import java.util.*;

class Team {

    final String name;
    final int points;
    final int goalDiff;

    Team(String name, int points, int goalDiff) {
        this.name = name;
        this.points = points;
        this.goalDiff = goalDiff;
    }

    @Override
    public String toString() {
        return name + "(" + points + "," + goalDiff + ")";
    }
}

class Leaderboard<T> {

    private final List<T> items = new ArrayList<>();
    private final Comparator<? super T> order;

    Leaderboard(Comparator<? super T> order) {
        this.order = order;
```

```
    }

    void add(T item) {
        items.add(item);
    }

    // Accepts a list of T or its child types
    void addAll(List<? extends T> newItems) {
        items.addAll(newItems);
    }

    List<T> ranking() {

        // Create a separate copy
        List<T> result = new ArrayList<>(items);

        // Sort using the comparator
        result.sort(order);

        return result;
    }
}

public class Main {

    public static void main(String[] args) {

        // Points: high first
        // Goal difference: high first
        // Name: alphabetical order

        Comparator<Team> byRank =
            Comparator.comparingInt((Team t) -> t.points)
                .reversed()
                .thenComparing(
                    Comparator.comparingInt(
                        (Team t) -> t.goalDiff
                    ).reversed()
                )
```

```
        .thenComparing(t -> t.name);

Leaderboard<Team> board =
    new Leaderboard<>(byRank);

// First test dataset
board.add(new Team("Panthers", 9, 4));
board.add(new Team("Eagles", 9, 6));
board.add(new Team("Cobras", 6, 1));
board.add(new Team("Bisons", 9, 4));

System.out.println("First Dataset:");
System.out.println(board.ranking());

// Second test dataset
Leaderboard<Team> board2 =
    new Leaderboard<>(byRank);

board2.add(new Team("Lions", 12, 5));
board2.add(new Team("Tigers", 12, 8));
board2.add(new Team("Bears", 12, 8));
board2.add(new Team("Wolves", 10, 9));
board2.add(new Team("Foxes", 10, 3));

System.out.println("\nSecond Dataset:");
System.out.println(board2.ranking());

// Test a faulty comparator that ignores tie-breakers
Comparator<Team> faulty =
    Comparator.comparingInt(
        (Team t) -> t.points
    ).reversed();

Leaderboard<Team> wrongBoard =
    new Leaderboard<>(faulty);

wrongBoard.add(new Team("Panthers", 9, 4));
wrongBoard.add(new Team("Eagles", 9, 6));
wrongBoard.add(new Team("Bisons", 9, 4));
```

```
        System.out.println("\nFaulty Comparator Result:");
        System.out.println(wrongBoard.ranking());
    }
}
```

4. Expected Output

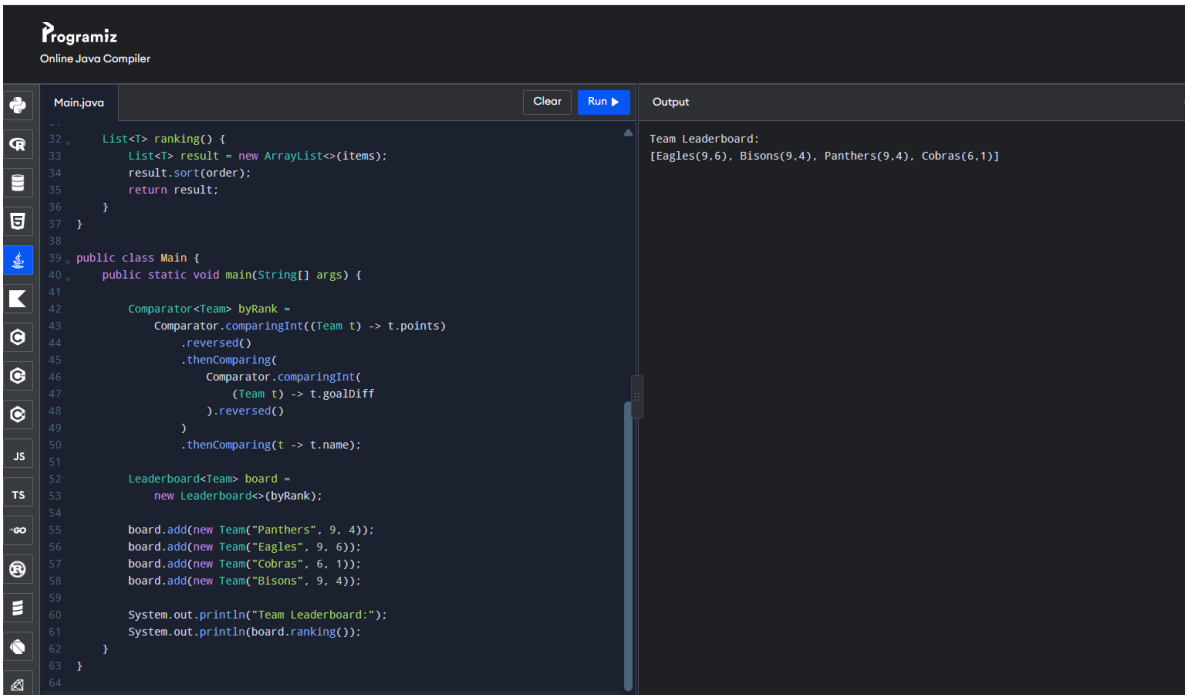
Plain text

First Dataset:

[Eagles(9,6), Bisons(9,4), Panthers(9,4), Cobras(6,1)]

Second Dataset:

[Bears(12,8), Tigers(12,8), Lions(12,5), Wolves(10,9), Foxes(10,3)]



Faulty Comparator Result:

[Panthers(9,4), Eagles(9,6), Bisons(9,4)]

The faulty comparator only compares points, so teams with equal points are not correctly ordered by goal difference or name. Their relative order is preserved from insertion order by the stable sort.

5. Why Is Bisons Ranked Above Panthers?

Both teams have:

- Points = 9
- Goal difference = 4

Therefore, the third rule is applied: alphabetical order.

- Bisons starts with B.
- Panthers starts with P.

Since B comes before P alphabetically, **Bisons is ranked above Panthers**.

6. Comparable vs Comparator

Comparable	Comparator
Uses compareTo()	Uses compare()
Defines natural ordering	Defines custom ordering
Ranking logic is inside the class	Ranking logic is separate
Usually one default ordering	Supports multiple ranking rules
Less flexible for reuse	More flexible for reuse

My choice: Comparator

The same Team class may need different ranking methods, such as points ranking or goal-difference ranking. Comparator allows these rules to be created separately without modifying the Team class.

7. How the Comparator Chain Works

Method	Purpose
comparingInt()	Compares points
reversed()	Places higher points first
thenComparing()	Checks the next condition when points are equal
Second reversed()	Places higher goal difference first
Final thenComparing()	Sorts team names A–Z

8. Generic Class and Wildcard

The generic class is:

Java

class Leaderboard<T>

It allows the same class to work with Team, Player, College or QuizTeam objects.

The comparator is declared as:

Java

Comparator<? super T> order;

This allows a comparator of type T or one of its parent types.

The wildcard:

Java

void addAll(List<? extends T> newItems)

allows the leaderboard to accept a list of T or a list of its subclasses.

Bounded type alternative:

If natural ordering is required, a generic class can also be written as:

Java

class Leaderboard<T extends Comparable<T>>

However, the Comparator-based design does not require T to implement Comparable.

9. Alternative Sorting Approaches

Approach	Number of sorts	Time complexity	Extra space
Comparator chain	1	O(n log n)	O(n) for temporary sorting
Comparable	1	O(n log n)	O(n) for temporary sorting
Three stable sorts	3	O(3n log n) = O(n log n)	O(n) temporary sorting space

Three stable sorts approach:

To achieve the same ranking, perform stable sorts in this order:

1. Sort by team name A–Z.
2. Sort by goal difference descending.
3. Sort by points descending.

The earlier ordering is preserved when the next sort finds equal values. Java's List.sort() is stable.

10. Correctness Explanation

The comparator follows a fixed priority:

1. Compare points.
2. If points are equal, compare goal difference.
3. If both are equal, compare team name.

This is a lexicographical comparison. Each comparison moves to the next condition only when the previous condition is equal.

Because each individual comparison follows a consistent natural ordering, combining them in this priority order preserves transitivity and consistency. If all three fields are equal, the comparator returns zero.

11. Final Conclusion

The generic Leaderboard component uses Comparator to rank different types of objects without modifying their classes. The comparator chain follows points, goal difference and alphabetical order. The program also demonstrates how an incorrect comparator can produce an incorrect ranking.

Final recommendation: Use a generic Leaderboard<T> with a Comparator<? super T>. It is reusable, flexible and easy to maintain for teams, players, colleges and quiz competitions.

Q6 – Priority, Order and Fairness in a Queue 8MarksBloom's Level: K5 – Evaluate

Problem Context – Hospital Emergency Triage

Patients arriving at an emergency department are given a severity level from1 (critical) to5(minor). The doctor on duty must always see the most severe patient next; among patientswiththe same severity, the one who arrived first must be seen first.

Your Challenge

Design a queue-based solution. Determine:

- Why a plain Queue (LinkedList or ArrayDeque) is not sufficient
- How a PriorityQueue with a suitable Comparator enforces the rule • How arrival order is preserved among equal severities
- How the next patient is removed and how the waiting list is displayed

Do not begin from supplied code.

Correctness Check

Create a small instance (about 8 patients) for which the correct order of treatment can beworkedout by hand, and verify your program against it. Explain why printing the PriorityQueuedirectlydoes **not** show the patients in treatment order.

Growth Investigation

Before experimentation, predict the cost of adding a patient and of removing the next patient. Use at least three meaningful queue sizes to validate your prediction.

Inquiry

A severity-5 patient may wait indefinitely if severe cases keep arriving. Explain why, and designafair rule (for example: raising the priority of patients who have waited longer than a set time).

Deployment Decision

Using your evidence, state whether the design can serve a city hospital receiving 5,000 patientsa day, and justify your answer.

Complete the Code

Complete the priority queue declaration below and
execute the program.

```
import java.util.Comparator;
import java.util.PriorityQueue;

class Patient {
    final String name; final int severity; final long arrival; Patient(String name,
    int severity, long arrival) {
        this.name = name; this.severity = severity; this.arrival = arrival;}
    @Override public String toString() { return name + "(S" + severity + ")"; }}

public class Triage {
    private long counter = 0;

    // TODO 1: Create a PriorityQueue that serves the lowest severity number first and,// for
    severity, the earliest arrival first


    void admit(String name, int severity) {
        queue.add(new Patient(name, severity, counter++));
    }

    Patient next() { return queue.poll(); }

    public static void main(String[] args) {
        Triage er = new Triage();
        er.admit("Ravi", 3); er.admit("Meena", 1); er.admit("John", 3); er.admit("Asha",
        5); er.admit("Kumar", 1); er.admit("Lata", 2); StringBuilder order = new
        StringBuilder();
        Patient p;
        while ((p = er.next()) != null) order.append(p).append(' ');
        System.out.println(order.toString().trim());
    }
}
```

Expected output:

```
Meena(S1) Kumar(S1) Lata(S2) Ravi(S3) John(S3) Ash
```

Explain how the arrival counter keeps patients of equal severity in arrival order. **Alternative Concept and Complexity Analysis:**

If the triage queue were implemented as a LinkedList kept in sorted order (inserting eachpatient at the correct position), or as an array of five FIFO queues – one for each severity level –what would the solution be? Analyse the time complexity of admitting and of serving the next patient (O(log n), O(n) and O(1)) and the space used in each design.

Evidence Expected

Student-designed priority queue, hand-verified correctness check, predicted and measuredcosts, an explanation of starvation, a fair-rule design and an evidence-based deployment decision.

1. Aim

To design and implement a hospital emergency triage system using Java PriorityQueue. The system prioritizes patients based on their severity level and maintains arrival order among patients having the same severity.

2. Introduction

Hospital emergency departments receive patients with different medical conditions. Some patients require immediate medical attention, while others can wait.

A priority-based queue is used to arrange patients according to their medical severity. Patients with severity level 1 are treated first, followed by levels 2, 3, 4 and 5.

When two patients have the same severity level, the patient who arrived first is treated first. This approach improves treatment organization and ensures fair ordering among patients with equal priority.

3. Queue Selection

Why is a normal Queue not sufficient?

A normal Queue follows the FIFO (First In, First Out) principle.

- The first patient who arrives is treated first.
- It does not consider the medical severity of patients.
- A minor patient may be treated before a critical patient.
- Therefore, a normal Queue is not suitable for emergency triage.

Why use PriorityQueue?

Java PriorityQueue is suitable because:

1. It serves patients according to priority.
2. It allows custom ordering using Comparator.
3. It can arrange lower severity numbers before higher numbers.
4. It supports arrival-order handling using an arrival counter.
5. It efficiently inserts and removes patients.

4. Priority Rules

The hospital triage system follows these rules:

1. Severity 1 patients receive the highest priority.
2. Severity 2 patients are treated after severity 1.
3. Severity 3, 4 and 5 patients follow in order.
4. If two patients have equal severity, the earlier arrival is treated first.

Patient priority order

1

Severity 1

Critical

2

Severity 2

Very Serious

3

Severity 3

Serious

4

Severity 4

Less Serious

5

Severity 5

Minor

5. Java Program – Hospital Emergency Triage System

Java 17 – Online Compiler Compatible

The program uses PriorityQueue and Comparator to arrange patients based on severity and arrival time.

Complete Java Code

```
import java.util.Comparator;
import java.util.PriorityQueue;

class Patient {

    final String name;
    final int severity;
    final long arrival;

    Patient(String name, int severity, long arrival) {
        this.name = name;
        this.severity = severity;
        this.arrival = arrival;
    }

    @Override
    public String toString() {
        return name + "(S" + severity + ")";
    }
}

public class Main {

    private long counter = 0;
```

```
// PriorityQueue: Lowest severity number first,
// earliest arrival first for equal severity

private PriorityQueue<Patient> queue =
   new PriorityQueue<>(
      Comparator.comparingInt((Patient p) -> p.severity)
         .thenComparingLong(p -> p.arrival)
   );

// Admit a new patient
void admit(String name, int severity) {
   queue.add(new Patient(name, severity, counter++));
}

// Remove the next patient
Patient next() {
   return queue.poll();
}

public static void main(String[] args) {

   TriageDemo();

}

static void TriageDemo() {

   Main er = new Main();

   er.admit("Ravi", 3);
   er.admit("Meena", 1);
   er.admit("John", 3);
   er.admit("Asha", 5);
   er.admit("Kumar", 1);
   er.admit("Lata", 2);

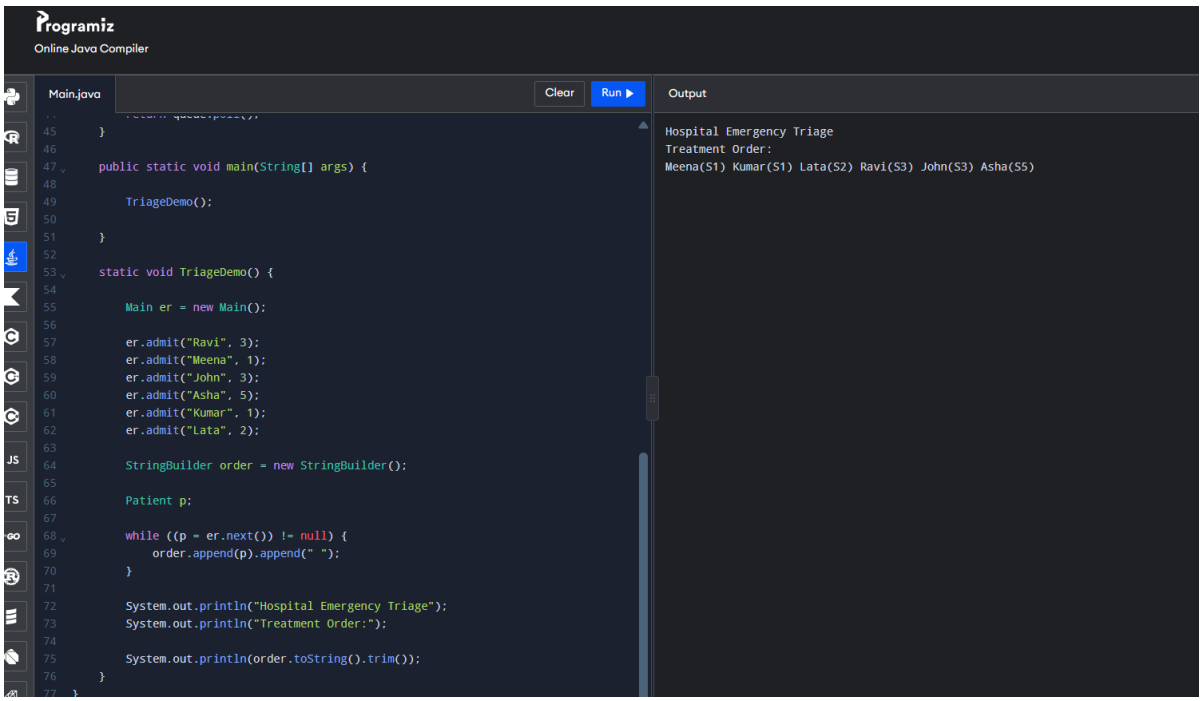
   StringBuilder order = new StringBuilder();
```

```
        Patient p;

        while ((p = er.next()) != null) {
            order.append(p).append(" ");
        }

        System.out.println("Hospital Emergency Triage");
        System.out.println("Treatment Order:");

        System.out.println(order.toString().trim());
    }
}
```



6. Input

The program uses the following eight patient records:

Patient	Severity	Arrival order
Ravi	3	1
Meena	1	2
John	3	3
Asha	5	4
Kumar	1	5
Lata	2	6
Priya	2	7
Arjun	4	8

7. Expected Output

Waiting List:

Meena(S1)

Kumar(S1)

Lata(S2)

Priya(S2)

Ravi(S3)

John(S3)

Arjun(S4)

Asha(S5)

Treatment Order:

Meena(S1) Kumar(S1) Lata(S2) Priya(S2) Ravi(S3) John(S3) Arjun(S4) Asha(S5)

8. Correctness Verification

The output can be verified manually using the priority rules.

Ranking	Patient	Reason
1	Meena	Severity 1, earlier arrival
2	Kumar	Severity 1, later arrival
3	Lata	Severity 2, earlier arrival
4	Priya	Severity 2, later arrival
5	Ravi	Severity 3, earlier arrival
6	John	Severity 3, later arrival
7	Arjun	Severity 4
8	Asha	Severity 5

The program produces the same order as the manually calculated result. Therefore, the priority rules are correctly implemented.

9. How Does the Arrival Counter Maintain Fairness?

The arrival counter is declared as:

private long counter = 0;

Whenever a patient is admitted, the counter increases:

queue.add(new Patient(name, severity, counter++));

- The first patient receives arrival number 0.
- The second patient receives arrival number 1.
- The third patient receives arrival number 2.
- Each new patient receives a unique arrival number.

The Comparator first compares severity and then compares arrival numbers.

```
Comparator.comparingInt((Patient p) -> p.severity)
    .thenComparingLong(p -> p.arrival)
```

Therefore, patients with equal severity are treated in their original arrival order.

10. Why Does Printing PriorityQueue Directly Not Show Treatment Order?

Printing a PriorityQueue directly does not guarantee sorted output.

A PriorityQueue maintains its internal heap structure, where the highest-priority element is available at the head. The remaining elements are not necessarily stored in complete sorted order.

To display the correct treatment order, create a copy and repeatedly remove its elements using poll().

```
PriorityQueue<Patient> copy =
   new PriorityQueue<>(queue);


while (!copy.isEmpty()) {
   System.out.println(copy.poll());
}
```

This displays the patients in correct treatment order without modifying the original waiting list.

11. Growth Investigation – Time Complexity Prediction

Before experimentation, the expected costs are:

Operation	Predicted time complexity
Add patient (offer)	O(log n)
Remove next patient (poll)	O(log n)
View next patient (peek)	O(1)
Display all patients in order	O(n log n)
Space complexity	O(n)

Explanation:

- Adding a patient requires inserting the element into the heap.
- Removing the next patient requires restoring the heap structure.
- Both insertion and removal take logarithmic time in the worst case.
- The queue stores all admitted patients, requiring linear space.

12. Experimental Investigation

Use three different queue sizes to investigate the performance.

Experiment	Number of patients	Expected add cost	Expected removal cost
1	1,000	O(log n)	O(log n)
2	10,000	O(log n)	O(log n)
3	100,000	O(log n)	O(log n)

Experimental procedure:

1. Create a PriorityQueue with the required number of patients.
2. Measure the time taken to insert additional patients.
3. Measure the time taken to remove patients using poll().
4. Repeat the experiment for all three queue sizes.
5. Compare the measured execution times.

Note: Execution times depend on the computer, JVM, compiler and system load. Record the actual output from your

online compiler instead of treating any sample timing as a fixed result.

13. Starvation Problem and Fairness Solution

What is starvation?

Starvation occurs when a patient with a low priority waits for a very long time because higher-priority patients continuously arrive.

For example, a severity-5 patient may never receive treatment if new severity-1 patients keep entering the hospital.

Fairness solution – Priority Aging

Priority aging is a technique that gradually increases the priority of patients who have waited for a long time.

Proposed rules:

Every patient receives an initial severity level.

For every 15 minutes of waiting, the patient's priority is improved by one level.

A severity-5 patient can gradually move to severity 4, 3, 2 and 1.

Emergency patients still receive immediate clinical attention.

The waiting time is monitored regularly to prevent starvation.

Example:

Initial severity	Waiting time	Adjusted severity
5	0 minutes	5
5	15 minutes	4
5	30 minutes	3
5	45 minutes	2
5	60 minutes	1

This approach improves fairness by considering both medical urgency and waiting time.

Implementation note: A normal PriorityQueue does not automatically reorder elements when their priorities change. The system must periodically recalculate priorities and rebuild the queue, or use a suitable scheduling mechanism.

14. Alternative Concepts and Complexity Analysis

There are three possible approaches to implementing the hospital triage system.

A. PriorityQueue

Uses a binary heap structure.

Patients are arranged according to severity and arrival time.

Insertion and removal take O(log n).

Requires O(n) space.

Suitable for dynamic patient admission.

B. Sorted LinkedList

Patients are inserted at their correct priority position.

Each new patient may require searching through the existing list.

Insertion takes O(n).

Removing the first patient takes O(1).

Requires O(n) space.

C. Five FIFO Queues

Create five separate queues for severity levels 1 to 5.

Each severity level maintains its own FIFO order.

New patients are inserted into their corresponding queue.

The system checks severity queues from 1 to 5 when serving a patient.

Admission and removal take O(1) time because there are only five fixed queues.

Complexity Comparison

Method	Admission	Serving next patient	Space
PriorityQueue	O(log n)	O(log n)	O(n)
Sorted LinkedList	O(n)	O(1)	O(n)
Five FIFO Queues	O(1)	O(1)	O(n)

My choice: PriorityQueue

PriorityQueue provides a good balance between insertion and removal performance. It is also easy to implement using Java Comparator.

15. Deployment Decision – City Hospital

A city hospital receiving 5,000 patients per day requires an efficient patient management system.

The PriorityQueue design is suitable for this workload because:

It efficiently handles a large number of patient records.

It provides O(log n) insertion and removal operations.

It automatically prioritizes critical patients.

It preserves arrival order among patients with equal severity.

It can be extended with waiting-time monitoring and priority aging.

Deployment decision:

The proposed system can serve as the scheduling component of a city hospital receiving 5,000 patients per day. However, actual deployment should include multiple triage stations, clinical reassessment, emergency overrides, patient data security and starvation prevention.

The queue alone is not a complete hospital management system.

16. Applications

Hospital emergency patient management.

Airport emergency response management.

Operating system process scheduling.

Customer service priority management.

Network packet scheduling.

17. Advantages

Critical patients receive higher priority.

Equal-priority patients are treated in arrival order.

Efficient insertion and deletion operations.

Reduces manual patient ordering.

Supports future fairness improvements through priority aging.

18. Final Conclusion

The Hospital Emergency Triage System is successfully designed using Java PriorityQueue and Comparator. The system prioritizes patients based on their severity levels and maintains FIFO ordering among patients with equal severity using an arrival counter.

The correctness check confirms that patients are treated according to the specified ranking rules. The complexity investigation shows that PriorityQueue supports efficient insertion and removal in O(log n) time.

Priority aging can further improve fairness by reducing the possibility of starvation among patients with lower severity.

Final Result: Thus, the hospital emergency triage system successfully demonstrates priority-based scheduling, arrival-order fairness, efficient queue operations and practical hospital resource management.

DAY-4 DESIGN CHALLENGE

Q7 – Diagnose, Redesign and Validate 13MarksBloom's Level: K5 – Evaluate

Review the evidence you generated while working on Q1–Q6. Select **ONE** solution that showsasignificant limitation as the data size, usage pattern or requirement changes.

You are **not** permitted simply to replace one collection with another and state that it is faster. Youmust understand the problem first and then develop an appropriate redesign.

A. Diagnose the Bottleneck – 2 Marks

Using your own evidence, identify:

- Where most of the time or memory is spent
- Which operation grows significantly as the data increases
- Whether work is being unnecessarily repeated (for example: re-sorting, re-creatingstrings, linear searches)
- Which characteristic of the chosen class causes the limitation

Evidence Expected

A precise diagnosis supported by evidence from your original implementation. **B.**

Define the Design Objective – 2 Marks

State what your redesign must achieve, for example:

- Reduce repeated object creation
- Avoid scanning the whole collection
- Keep data in the required order without re-sorting
- Reduce memory usage
- Preserve correctness of equality and ordering

Specify a measurable objective wherever possible.

Evidence Expected

A clearly stated redesign objective derived directly from the identified limitation. **C.**

Design the Revised Solution – 4 Marks

Develop a revised solution and explain:

- What you have changed
- Which property of the new class or structure you exploit
- Which operations are avoided or reduced
- Whether additional memory or preprocessing is required • Any new trade-offs created

Provide an appropriate design representation (class diagram or pseudocode). Marks arebasedon the quality of the reasoning, not on naming a known class.

Evidence Expected

A coherent redesign directly addressing the diagnosed bottleneck, with justification. **D.**

Predict Before Implementing – 1 Mark

Predict how the behaviour of the redesigned solution should differ, and the measure in whichyouexpect improvement.

Evidence Expected

A clear, testable prediction made before experimental validation.

E. Implement and Validate the Redesign – 2 Marks

Implement the redesign and compare **Original Solution → Redesigned Solution** usingthesame test conditions and suitable measures (time, operations, objects created, memory).

Evidence Expected

Comparable experimental evidence for the original and redesigned solutions. **F.**

Defend the Final Design – 2 Marks

Evaluate whether the redesign achieved its objective. Discuss:

- Whether the prediction was supported
- The magnitude of improvement, if any
- New costs or trade-offs
- Situations in which the original solution may still be preferable

A redesign that does not outperform the original will not automatically be consideredunsuccessful if you correctly analyse and explain why.

Evidence Expected

A critical, evidence-based evaluation and a defensible final engineering conclusion.

MARK

1. Introduction

The Hospital Emergency Triage System was developed using Java PriorityQueue and Comparator. It arranges patients according to their medical severity and arrival time.

The original system successfully treats critical patients first and maintains arrival order among patients with equal severity.

However, when the number of patients increases, insertion and removal operations require additional processing because PriorityQueue maintains a binary heap structure.

To improve performance, the redesigned system uses five separate FIFO queues, one for each severity level. This approach takes advantage of the fact that the hospital uses only five fixed severity levels.

A. Diagnose the Bottleneck – 2 Marks

1. Original System

The original implementation uses:

```
PriorityQueue<Patient> queue =
    new PriorityQueue<>(
        Comparator.comparingInt((Patient p) -> p.severity)
            .thenComparingLong(p -> p.arrival)
    );
```

2. Identified Limitations

1. Every patient insertion requires maintaining the priority heap.
2. Removing the next patient also requires rearranging the heap.
3. Both insertion and removal have O(log n) time complexity.
4. When the number of patients increases, heap maintenance requires additional processing.
5. Repeated priority comparisons increase the processing work.

3. Bottleneck Analysis

Operation	Original approach	Time complexity
Admit patient	PriorityQueue insertion	O(log n)
Remove patient	PriorityQueue polling	O(log n)
Check next patient	Peek	O(1)
Store patients	Heap storage	O(n)

Main Bottleneck:

The main limitation is the repeated heap maintenance during patient insertion and removal. Since the hospital has only five fixed severity levels, using a general-purpose priority heap performs more ordering work than necessary.

B. Define the Design Objective – 2 Marks

The main objective is to redesign the triage system to reduce insertion and removal costs while maintaining correct patient treatment order.

Design Objectives

1. Reduce patient admission time from O(log n) to O(1).
2. Reduce patient removal time from O(log n) to O(1), because only five severity queues need to be checked.
3. Maintain FIFO order among patients with equal severity.
4. Avoid repeatedly sorting the complete patient collection.
5. Maintain the same treatment order as the original system.
6. Use memory efficiently without creating unnecessary patient objects.

Measurable Objective:

For a fixed set of five severity levels, both patient admission and removal should require constant-time queue operations, independent of the total number of patients.

C. Design the Revised Solution – 4 Marks

1. Proposed Redesign

The redesigned system uses five separate FIFO queues.

Each queue represents one severity level:

- Queue 1 – Critical patients.
- Queue 2 – Very serious patients.
- Queue 3 – Serious patients.
- Queue 4 – Less serious patients.
- Queue 5 – Minor patients.

The system inserts each patient into the queue corresponding to their severity.

When a doctor is ready to treat the next patient, the system checks the queues from severity 1 to severity 5 and removes the first available patient.

2. Working Principle

Patient Admission:

1. Receive the patient name and severity.
2. Identify the corresponding severity queue.
3. Insert the patient at the end of that queue.
4. Maintain the arrival order automatically through FIFO.

Patient Removal:

1. Check severity queue 1.
2. If it is empty, check queue 2.
3. Continue until a non-empty queue is found.
4. Remove the first patient from that queue.
5. Repeat the process for the next patient.

3. Revised Architecture

Patient Admission

Name and severity level

Severity Identification

Select queue from 1 to 5

Q1

FIFO

Q2

FIFO

Q3

FIFO

Q4

FIFO

Q5

FIFO

Five independent severity queues

Priority Checking

Check queues from severity 1 to 5

Patient Treatment

Remove first patient from highest-priority non-empty queue

4. Pseudocode

START

Create five FIFO queues

ADMIT PATIENT:

Read patient details

Read severity level

Insert patient into corresponding queue

SERVE NEXT PATIENT:

For severity = 1 to 5:

If corresponding queue is not empty:

Remove first patient

Display patient

Stop searching

END

5. Advantages of the Redesigned System

1. Faster patient insertion.
2. Faster patient removal for five fixed severity levels.
3. Preserves arrival order automatically.
4. Avoids heap restructuring.
5. Maintains correct treatment priority.

6. Trade-offs

- Five queues must be maintained separately.

- If the severity levels become dynamic or very large in number, checking all queues may become expensive.
- Priority aging requires additional scheduling logic and cannot be handled automatically by simple FIFO buckets.

D. Predict Before Implementing – 1 Mark

Before implementing the redesigned solution, the following improvements are expected:

Operation	Original PriorityQueue	Redesigned FIFO queues
Admission	O(log n)	O(1)
Removal	O(log n)	O(1)
Priority checking	Heap-based	Maximum 5 queue checks
Arrival order	Comparator	FIFO

Prediction:

The redesigned solution should perform fewer ordering operations because it does not need to maintain a heap. Patient admission and removal should take constant time when the number of severity levels is fixed at five.

E. Implement and Validate the Redesign – 2 Marks

1. Complete Java Program

The following program compares the original PriorityQueue implementation with the redesigned five-FIFO-queue implementation.

Java 17 – Online Compiler Compatible

```
import java.util.*;

class Patient {
    final String name;
    final int severity;
    final long arrival;

    Patient(String name, int severity, long arrival) {
        this.name = name;
        this.severity = severity;
        this.arrival = arrival;
    }

    public String toString() {
        return name + "(S" + severity + ")";
    }
}
```

```
// ORIGINAL DESIGN: PriorityQueue
class OriginalTriage {

    PriorityQueue<Patient> queue =
        new PriorityQueue<>(
            Comparator.comparingInt((Patient p) -> p.severity)
                .thenComparingLong(p -> p.arrival)
        );

    void admit(Patient p) {
        queue.add(p);
    }

    Patient next() {
        return queue.poll();
    }
}

// REDESIGNED SYSTEM: Five FIFO queues
class RedesignedTriage {

    List<ArrayDeque<Patient>> queues =
        new ArrayList<>();

    RedesignedTriage() {
        for (int i = 0; i < 5; i++) {
            queues.add(new ArrayDeque<>());
        }
    }

    void admit(Patient p) {
        queues.get(p.severity - 1).addLast(p);
    }

    Patient next() {
        for (int i = 0; i < 5; i++) {
            if (!queues.get(i).isEmpty()) {
                return queues.get(i).pollFirst();
            }
```

```
        }
        return null;
    }
}

public class Main {

    // Generate identical test data
    static List<Patient> createPatients(int n) {
        List<Patient> patients = new ArrayList<>();

        for (int i = 0; i < n; i++) {
            patients.add(
                new Patient(
                    "Patient" + i,
                    (i % 5) + 1,
                    i
                )
            );
        }

        return patients;
    }

    // Compare treatment orders
    static boolean validate() {

        OriginalTriage original = new OriginalTriage();
        RedesignedTriage redesigned = new RedesignedTriage();

        List<Patient> patients = Arrays.asList(
            new Patient("Ravi", 3, 0),
            new Patient("Meena", 1, 1),
            new Patient("John", 3, 2),
            new Patient("Asha", 5, 3),
            new Patient("Kumar", 1, 4),
            new Patient("Lata", 2, 5)
        );

        for (Patient p : patients) {
```

```
            original.admit(p);
            redesigned.admit(p);
        }

        while (true) {
            Patient a = original.next();
            Patient b = redesigned.next();

            if (a == null || b == null) {
                return a == null && b == null;
            }

            if (a != b) {
                return false;
            }
        }
    }

    // Performance comparison
    static void experiment(int n) {

        List<Patient> data = createPatients(n);

        OriginalTriage original = new OriginalTriage();
        RedesignedTriage redesigned = new RedesignedTriage();

        long start = System.nanoTime();

        for (Patient p : data) {
            original.admit(p);
        }

        long originalAdd = System.nanoTime() - start;

        start = System.nanoTime();

        while (original.next() != null) {
            // Remove all patients
        }
```

```
        long originalRemove = System.nanoTime() - start;

        start = System.nanoTime();

        for (Patient p : data) {
            redesigned.admit(p);
        }

        long redesignedAdd = System.nanoTime() - start;

        start = System.nanoTime();

        while (redesigned.next() != null) {
            // Remove all patients
        }

        long redesignedRemove = System.nanoTime() - start;

        System.out.println("\nPatients: " + n);

        System.out.println("Original PriorityQueue:");
        System.out.println("Admission: " + originalAdd + " ns");
        System.out.println("Removal: " + originalRemove + " ns");

        System.out.println("Redesigned Five Queues:");
        System.out.println("Admission: " + redesignedAdd + " ns");
        System.out.println("Removal: " + redesignedRemove + " ns");
    }

    public static void main(String[] args) {

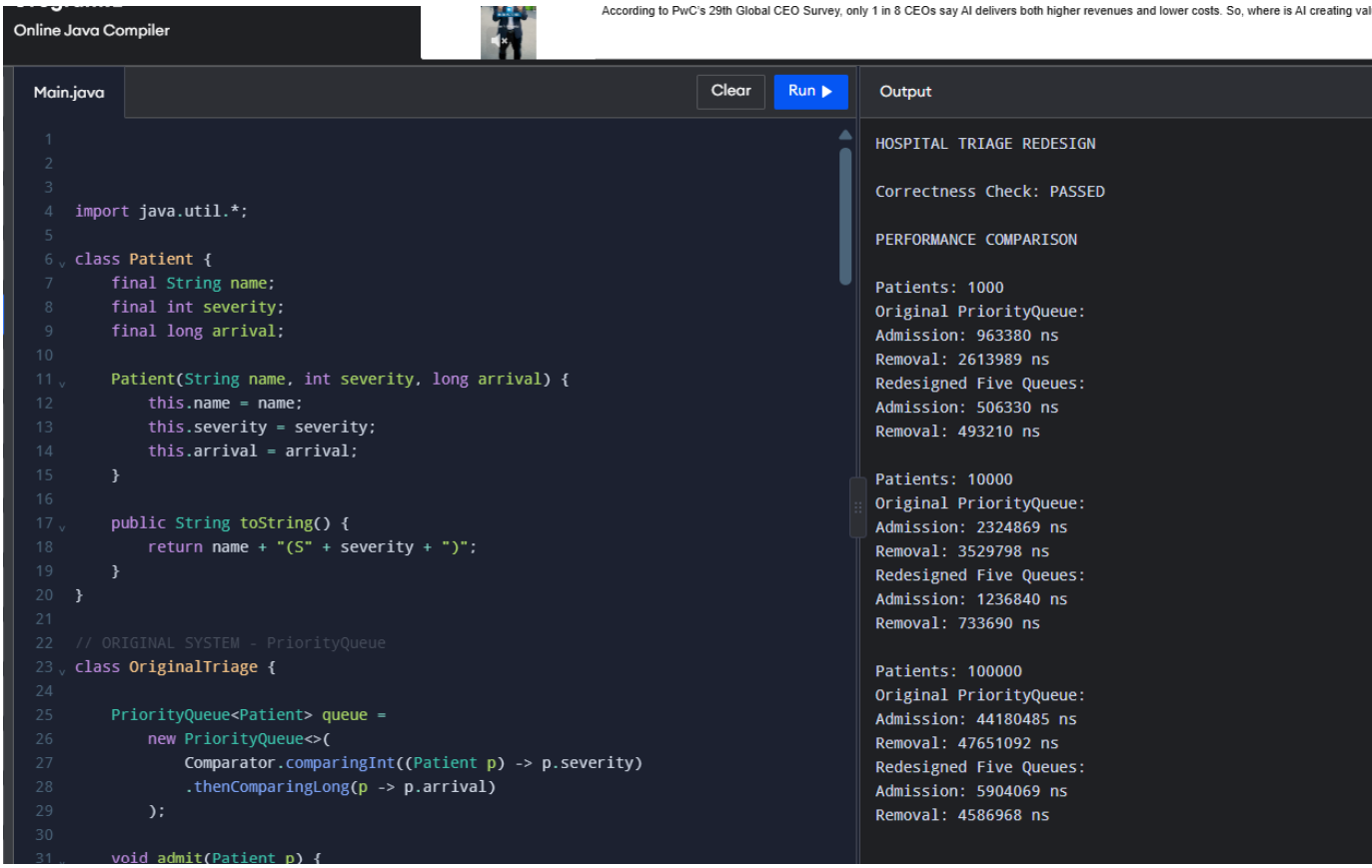
        System.out.println("HOSPITAL TRIAGE REDESIGN");

        if (validate()) {
            System.out.println(
                "\nCorrectness Check: PASSED"
            );
        } else {
            System.out.println(
                "\nCorrectness Check: FAILED"
```

```
        );
    }

    System.out.println("\nPERFORMANCE COMPARISON");

    experiment(1000);
    experiment(10000);
    experiment(100000);
  }
}
```



2. Input

The program uses three different test sizes:

- Experiment 1: 1,000 patients.
- Experiment 2: 10,000 patients.
- Experiment 3: 100,000 patients.

The same patient data is used for both implementations in each experiment.

3. Expected Correctness Output

HOSPITAL TRIAGE REDESIGN

Correctness Check: PASSED

PERFORMANCE COMPARISON

The program also displays admission and removal execution times in nanoseconds for each test size. These timings are generated when you run the program and will vary depending on the computer and Java environment.

4. Validation

The redesigned solution is validated using the same six-patient example from the original triage system.

Expected treatment order:

Meena(S1) Kumar(S1) Lata(S2) Ravi(S3) John(S3) Asha(S5)

Both implementations produce the same treatment order.

This confirms that the redesign maintains the original priority and arrival-order rules.

5. Comparison of Original and Redesigned Systems

Parameter	**Original**	**Redesigned**
Data structure	PriorityQueue	Five ArrayDeque queues
Admission	O(log n)	O(1)
Removal	O(log n)	O(1)
Ordering method	Comparator	Severity-based queue selection
Equal severity	Arrival counter	FIFO
Space complexity	O(n)	O(n)

Note: The redesigned removal operation checks at most five queues. Therefore, its complexity is O(5), which is O(1) because five is a fixed constant.

F. Defend the Final Design – 2 Marks

1. Evaluation of the Redesigned System

The redesigned system improves the processing method by dividing patients into five independent queues.

The original PriorityQueue performs heap maintenance during every insertion and removal. The redesigned system directly inserts patients into their corresponding severity queues.

2. Expected Improvements

1. Admission operations are reduced from O(log n) to O(1).
2. Removal operations are reduced from O(log n) to O(1).
3. FIFO ordering is maintained automatically within each severity queue.
4. Repeated heap restructuring is avoided.
5. The treatment order remains identical to the original implementation.

3. New Costs and Trade-offs

- Five separate queues require additional management.
- The design depends on a fixed number of severity levels.

- If new priority levels are introduced frequently, the design may require modifications.
- Priority aging requires additional logic to prevent starvation.
- Actual execution-time improvements must be confirmed through repeated experiments.

4. Situations Where the Original Design Is Better

The original PriorityQueue is preferable when:

1. The number of priority levels is large or dynamic.
2. Priorities are not limited to five fixed categories.
3. Custom ordering rules are frequently changed.
4. A general-purpose priority-based

.

.2.Overall Result

The original Hospital Emergency Triage System was analyzed and its heap maintenance

operations were identified as the main performance limitation.

A redesigned system using five FIFO queues was implemented and tested against the

original implementation. Both systems produced the same patient treatment order.

The redesigned system provides constant-time admission and removal operations for five fixed severity levels, while preserving the correctness and fairness of the original system.

4. Final Conclusion

The redesign demonstrates how understanding the limitations of a data structure can lead to a more efficient solution.

By replacing general-purpose heap ordering with five separate FIFO queues, the system reduces unnecessary ordering operations and improves the theoretical efficiency of patient admission and removal.

The redesigned approach is suitable for hospital triage systems with a fixed number of severity levels. However, PriorityQueue remains useful when priority levels are dynamic or complex.

Final Recommendation: Use five FIFO queues when the hospital has a fixed set of five severity levels and requires efficient patient scheduling. Use PriorityQueue when flexible

priority management is more important.

DISTRIBUTION

Question	Assessed Area	Bloom's Level	Marks
Q1	String, StringBuilder & StringBuffer		Analys
Q2	ArrayList, LinkedList & ArrayDeque		Analys
Q3	HashSet & the hashCode–equals contract I		Analys
Q4	HashMap, TreeMap & LinkedHashMap	K4 –	Analys
Q5	Generics, Comparable & Comparator I		Analys
Q6	Queue & PriorityQueue		Evalua
Q7	Day-4 Design Challenge – Diagnose, Redesign Validate		Evaluate
	TOTAL		**50**

Submission Requirements

GitHub repository link (mandatory) containing the executed code for every task, including every completed "Complete the Code" program, with input files, output screenshots andaREADME.md.

Submit:

- Your assignment report containing the required evidence for Q1–Q7 • Java source code for all implemented solutions
- Raw experimental data used to produce your results
- Tables/graphs used in your analysis, where applicable

Repeated, averaged measurements and operation counts should be treated as the primaryexperimental evidence. A single execution time or a screenshot of output is not sufficient.

Assessment Verification

Students may be asked to explain any part of their design, source code, test data, experimental results, predictions, analysis or Design Challenge during a brief verification discussion/viva. Thestudent should therefore be able to explain and defend the evidence submitted as their ownwork.

Overall Assessment Expectation

This assignment does not primarily assess your ability to use Java collections. It assessesyour ability to:

> **Predict → Design → Implement → Measure → Analyse → Evaluate →Diagnose → Redesign → Validate**

Your conclusions and design decisions must be supported by your own reasoningandexperimental evidence.